

BUSAM: A REAL-TIME BUS TRACKING AND DIGITAL FARE MANAGEMENT SYSTEM

BY

RAJ GURUNG

T.U. REGISTRATION NO.: [7-2-551-61-2023]

SYMBOL NO.: [16033/23]

NATIONAL COLLEGE OF COMPUTER STUDIES

A Project Report Submitted To

Faculty of Management, Tribhuvan University

in partial fulfilment of the requirements for the degree of

Bachelor of Information Management (BIM)

Kathmandu

September, 2026

STUDENT DECLARATION

This is to certify that I have completed the Project entitled "BusAm: A Real-Time Bus Tracking and Digital Fare Management System" under the guidance of “Radha Krishna Gajurel” in partial fulfilment of the requirements for the degree of Bachelor of Information Management at Faculty of Management, Tribhuvan University. This is my original work, and I have not submitted it earlier elsewhere.

Date: 2026-09-09

Signature:

Name: Raj Gurung

CERTIFICATE FROM THE SUPERVISOR

This is to certify that the project entitled "**BusAm: A Real-Time Bus Tracking and Digital Fare Management System**" is an academic work done by "**Raj Gurung**" submitted in the partial fulfilment of the requirements for the degree of Bachelor of Information Management at Faculty of Management, Tribhuvan University under my guidance and supervision. To the best of my knowledge, the information presented by him/her in the project report has not been submitted earlier.

Signature of the Supervisor

Name: Radha Krishna Gajurel

Designation Date: date

APPROVAL SHEET

This is to certify that the project titled BusAm: A Real-Time Bus Tracking and Digital Fare Management System submitted by [Your Full Name] has been examined and approved. In our opinion, it meets the required scope and quality standards for a project submitted in partial fulfillment of the requirements for the degree of Bachelor of Information Management (BIM).

Approval Panel:

S.No.	Name	Designation	Signature
1	Radha Krishna Gajurel	Project Supervisor	
2	Rajan Poudel	Program Coordinator	

Date of Defense: 9th September 2026

Department: Bachelor of Information Management (BIM)

Faculty: Management

ACKNOWLEDGEMENTS

I would like to extend my heartfelt gratitude to Mr. Radha Krishna Gajurel, my project supervisor, for his invaluable guidance, motivation, and insightful suggestions throughout the planning, development, and deployment of this project. His continuous support played a vital role in the successful completion of this work.

I am also sincerely thankful to the faculty members of the National College of Computer Studies for their valuable knowledge and guidance throughout my academic journey. I would like to thank my family and friends for their constant encouragement, patience, and understanding during this project.

Lastly, I would like to acknowledge my own dedication, perseverance, and determination, which enabled me to successfully complete this project despite numerous technical challenges encountered along the way.

Raj Gurung

Symbol No.:16033/23

T.U. Registration No.: 7-2-551-61-2023

September,2026

ABSTRACT

BusAm is a real-time public bus tracking and digital fare management system designed to modernize Kathmandu's public transportation experience. Developed using Flutter for the mobile front end and FastAPI (Python) for the backend, the system connects a driver application and a passenger application through a persistent WebSocket relay, allowing passengers to see a bus's live location on an interactive map. The platform also introduces a QR-code-based digital fare system: passengers scan a code when boarding and again when alighting, and the system calculates a distance-based fare using the Haversine formula and deducts it from an in-app wallet. Routes are modelled as ordered lists of stops, enabling a bus company/route selection flow for drivers and a From/To route-search feature for passengers. The backend is deployed on Render, giving the system a fixed public endpoint reachable from any network. The project demonstrates the practical application of real-time client-server communication, geolocation processing, and mobile application development to solve a real transportation problem faced by commuters in Kathmandu.

Table of Contents

STUDENT DECLARATION.....	ii
CERTIFICATE FROM THE SUPERVISOR	iii
APPROVAL SHEET	iv
ACKNOWLEDGEMENTS	v
ABSTRACT.....	vi
LIST OF FIGURES	ix
LIST OF TABLES	x
LIST OF ABBREVIATIONS	xi
CHAPTER I: INTRODUCTION	1
1.1 Background of the Project	1
1.2 Problem Statement.....	1
1.3 Objectives	1
1.4 Review of Related Work and Literature.....	2
1.5 Development Methodology.....	3
1.6 Scope and Limitations	4
1.6.1 Scope.....	4
1.6.2 Limitation	5
1.7 Report Organization.....	5
Chapter I: Introduction.....	5
Chapter II: System Development Process	5
Chapter III: Conclusion and Recommendation.....	5
CHAPTER II: SYSTEM DEVELOPMENT PROCESS	6
2.1 Analysis	6
2.1.1 Requirement Analysis.....	6
2.1.2 Feasibility Study.....	8
i. Technical Feasibility	8
ii. Operational Feasibility	8
iii. Economic Feasibility.....	8
iv. Schedule Feasibility	8
2.1.3 System Modelling.....	9

i. Object Modelling using Class and Object Diagrams	9
ii. Dynamic Modelling using State and Sequence Diagrams	11
iii. Process Modelling using Activity Diagram	14
2.3 Implementation	19
2.3.1 Tools Used.....	19
2.3.2 Implementation Detail of Modules	19
i. Role Selection Module	19
ii. Live GPS Broadcasting Module (Driver)	19
iii. WebSocket Relay Module (Backend).....	20
iv. Live Map and Route Search Module (Passenger).....	20
v. QR-Based Fare Module	20
vi. Wallet Module	20
vii. Deployment Module.....	21
2.3.3 Testing.....	21
CHAPTER III: CONCLUSION AND RECOMMENDATION.....	23
3.1 Summary.....	23
3.2 Conclusion	23
3.3 Recommendation.....	23
Short-term	23
Medium-term.....	23
Long-term	24
REFERENCE	25
APPENDIX.....	a

LIST OF FIGURES

Figure 1.1: Iterative Development Methodology

Figure 2.1: Use Case Diagram

Figure 2.2: Gantt Chart

Figure 2.3: Class Diagram

Figure 2.4: Object Diagram

Figure 2.5: Sequence Diagram of Driver GPS Broadcasting

Figure 2.6: Sequence Diagram of QR-Based Boarding and Exit (Fare Calculation)

Figure 2.7: Passenger State Diagram

Figure 2.8: Activity Diagram of the Live Tracking and Fare Process

Figure 2.9: Refinement of Class Diagram

Figure 2.10: Component Diagram

Figure 2.11: Deployment Diagram

LIST OF TABLES

Table 2.1: Unit Testing

Table 2.2: Integration Testing

Table 2.3: System Testing

LIST OF ABBREVIATIONS

API: Application Programming Interface

APK: Android Package Kit

BIM: Bachelor of Information Management

CASE: Computer-Aided Software Engineering

FPS: Frames Per Second (n/a) – removed

GPS: Global Positioning System

HTTP/HTTPS: HyperText Transfer Protocol (Secure)

JSON: JavaScript Object Notation

JWT: JSON Web Token

KM: Kilometre

NCCS: National College of Computer Studies

NPR: Nepalese Rupee

QR: Quick Response (Code)

REST: Representational State Transfer

SDK: Software Development Kit

UML: Unified Modeling Language

URL: Uniform Resource Locator

UUID: Universally Unique Identifier

WSGI: Web Server Gateway Interface

WSS: WebSocket Secure

CHAPTER I: INTRODUCTION

1.1 Background of the Project

Public bus transportation in Kathmandu remains one of the most widely used modes of travel, yet it continues to rely on largely manual, uncoordinated systems. Passengers have no reliable way of knowing where a bus currently is, how long it will take to arrive, or whether a bus travelling a given route also passes through their intended stop. Fare collection is almost entirely cash-based, which is slow, error-prone, and inconvenient for both passengers and conductors. As smartphone penetration and mobile data access have grown rapidly across Nepal, similar problems in ride-hailing and delivery services (such as Pathao and inDrive) have already been solved through real-time GPS tracking and digital wallets. BusAm applies the same underlying principles — live location broadcasting, digital fare calculation, and a simple mobile interface — specifically to the fixed-route public bus system.

The project was developed as a sixth-semester BIM project at the National College of Computer Studies, Tribhuvan University, under the supervision of Radha Krishna Gajurel. It combines a Flutter-based mobile front end with a FastAPI backend, using WebSockets for real-time communication between a driver application and a passenger application, and a QR-code-based fare mechanism for automated, distance-based fare deduction.

1.2 Problem Statement

Commuters using Kathmandu's public bus system currently face several recurring problems: they cannot see where a bus currently is or estimate its arrival time, they have no simple way to check which buses travel between two specific stops, and fare payment depends entirely on cash handled manually by a conductor, which is slow, hard to audit, and inconvenient for both passengers and operators. There is no existing, centralized digital system that solves these three problems together for Nepal's fixed-route bus network. BusAm addresses this gap by providing live GPS-based bus tracking, a route-search feature built on top of predefined stop sequences, and an automated, distance-based digital fare system driven by QR-code scans.

1.3 Objectives

- To develop a real-time GPS tracking system that allows passengers to view a bus's live location on a map.
- To design a route-matching mechanism that lets passengers search for buses travelling between a chosen start and end stop.

1.4 Review of Related Work and Literature

The transition from traditional cash-based public transportation to automated fare collection has been widely studied as part of the development of intelligent transportation systems. Automated fare collection (AFC) systems are designed to simplify fare payment, reduce manual cash handling, improve transaction accuracy, and generate digital records of passenger activity. Bieler et al. (2022) reviewed automated fare collection solutions and highlighted the increasing importance of mobile devices in public transportation ticketing. Their study found that mobile-based ticketing can make public transportation more seamless while also generating useful transactional data for transportation operators. The researchers also identified mobile technologies, sensor data, and automated validation as important areas for the future development of public transportation systems.

Pelletier et al. (2011) conducted a comprehensive review of smart card data use in public transportation. Their research explains that automated fare collection systems not only simplify revenue collection but also generate large amounts of information about passenger transactions. Such information can be used for operational management, service planning, ridership analysis, and understanding passenger travel patterns. Smart card systems can therefore provide benefits beyond payment by allowing transportation authorities to make better decisions using historical transaction data. This concept is relevant to the proposed system because digital records of fares and journeys can be maintained and later used for monitoring and analysis.

Farooqi et al. (2018) further examined the applications of transit smart cards beyond their original purpose of fare collection. Their literature review identified several uses of automated fare collection data, including origin-destination estimation, travel pattern analysis, trip-purpose detection, route-choice modelling, and transit performance measurement. The study demonstrates that digital fare systems can become an important source of transportation information rather than functioning only as payment mechanisms. The proposed Digital Bus Pass and Fare Management System similarly maintains digital records of passenger payments and rides, which can provide useful information for future transportation analysis.

Mobile ticketing has also been investigated as an alternative to conventional physical tickets and smart cards. The MobiPag system presented by Rinne et al. (2014) explored an integrated mobile payment and ticketing solution using near-field communication (NFC). The system allowed users to purchase bus tickets through a mobile device, store tickets electronically, validate them during travel, and review transaction information. This work demonstrates how mobile devices can combine payment, ticket storage, validation, and transaction management within a single platform. Although the proposed

BusAm system uses QR-based validation rather than NFC, it follows a similar principle of replacing physical tickets with digitally managed transportation services.

QR-code-based ticket validation has also been explored as a practical approach for public transportation. The Beep4Me system studied by Buda et al. (2022) investigated automatic ticket validation using smartphone sensors and QR codes. In the existing configuration described in the study, passengers could purchase tickets through a mobile application and scan a QR code placed inside a vehicle to validate their journey. The research demonstrates the feasibility of combining smartphones, QR codes, GPS, Bluetooth, and other device sensors to simplify the passenger validation process. This approach is particularly relevant to the proposed system because BusAm also uses QR-based interaction between passengers and buses for digital fare processing.

Real-time vehicle tracking is another important component of modern public transportation systems. Corsar et al. (2017) developed the GetThere system to provide real-time passenger information by integrating transportation data with GPS-based vehicle locations and information contributed through mobile devices. The system allowed users to view available bus services and obtain information about vehicle locations. The research demonstrated that real-time passenger information systems can integrate different sources of transportation data to provide passengers with useful journey information. This supports the use of GPS-based bus tracking in the proposed system, where passengers can view nearby buses and obtain their current locations through a mobile application.

1.5 Development Methodology

The BusAm project was developed using the Incremental development model, in which the system is built and delivered in small, functional segments rather than as a single complete unit at the end of the development period. Each increment builds upon the previous one, adding new functionality while ensuring that the core system remains stable and testable throughout the development process. This approach was chosen because the project's requirements evolved naturally as development progressed, for instance, the initial increment focused solely on establishing real-time GPS communication between the driver and passenger applications, after which subsequent increments introduced route searching, QR-based fare collection, digital wallet functionality, ride passes, and finally user authentication with database persistence. Each increment was designed, implemented, tested, and deployed to a live server before the next increment began, allowing issues to be identified and corrected early rather than accumulating until a final integration phase. This iterative addition of features also allowed the system architecture to adapt as new requirements were identified, such as introducing a PostgreSQL database only once user authentication became necessary,

rather than over-engineering the system with unused infrastructure from the outset. The incremental model was particularly well suited to this project given the limited development timeframe and the need to maintain a consistently working, demonstrable application at every stage rather than risking a non-functional system until the very end.

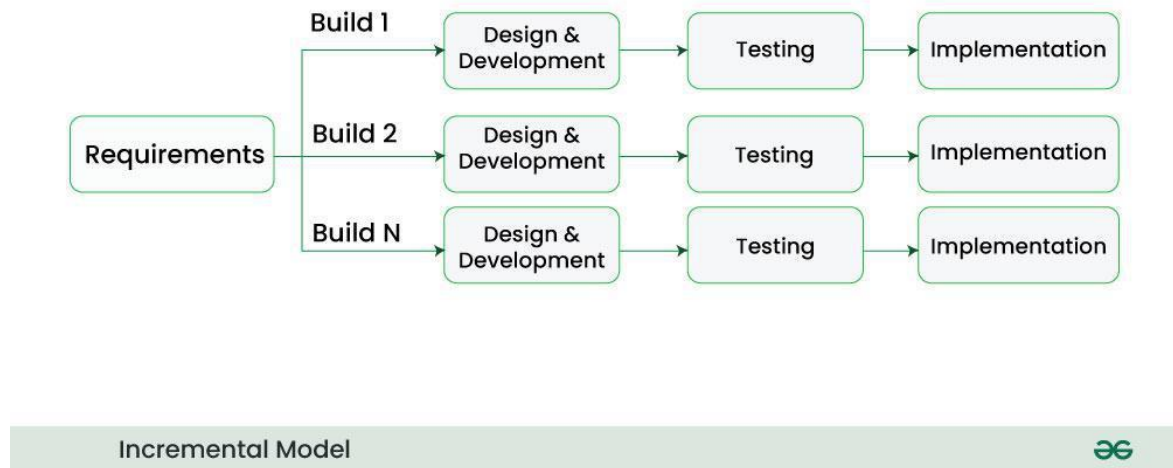


Figure 1.1: Incremental Development Methodology

1.6 Scope and Limitations

1.6.1 Scope

- Role-based mobile application with separate Driver and Passenger flows, launched from a single role-selection screen.
- Real-time GPS broadcasting from the driver's device to all connected passenger devices using a WebSocket relay.
- Live map display (OpenStreetMap tiles via flutter_map) showing the bus's current position and the passenger's own position.
- Route modelling as ordered lists of named stops, with a company/route selection flow for drivers and a From/To autocomplete search for passengers.
- QR-code-based boarding and exit scanning, with automatic distance calculation (Haversine formula) and tiered fare deduction from an in-app wallet.
- Backend deployment to a public cloud host (Render), giving the system one fixed URL reachable from any network.

1.6.2 Limitation.

- No user authentication/login system exists yet; each device is identified only by a locally generated device ID.
- Only a small, hardcoded set of routes and stops is supported; there is no admin interface for managing routes.
- Fare distance is calculated as a straight-line (Haversine) distance between boarding and exit points rather than the actual road distance travelled..
- The system depends on an active internet connection on both the driver's and the passenger's device; there is no offline mode.

1.7 Report Organization

Chapter I: Introduction

This chapter introduces the background, problem statement, and objectives of BusAm, reviews related transportation and fare-tracking platforms, describes the Iterative Development Methodology used, and defines the project's scope and limitations.

Chapter II: System Development Process

This chapter documents the complete development lifecycle: requirement analysis, feasibility study, object-oriented system modelling, system design, implementation details of each module, and the testing performed on the system.

Chapter III: Conclusion and Recommendation

This chapter summarizes what was achieved, reflects on the outcome of the project, and provides recommendations for future enhancement, including database integration, authentication, and completion of the payment gateway.

CHAPTER II: SYSTEM DEVELOPMENT PROCESS

2.1 Analysis

BusAm is a mobile and backend system built using Flutter for the client applications and FastAPI (Python) for the server. It uses a WebSocket-based publish-subscribe architecture to relay GPS data from a driver's device to all connected passenger devices in real time, and a REST-style HTTP endpoint for QR-based fare calculation and wallet management. The following sections detail the analysis, design, and implementation carried out to build the system.

2.1.1 Requirement Analysis

i. Functional Requirement

- Driver location broadcast: the driver's screen must read the device's live GPS position and send it to the backend at regular intervals while "Go Online" is active.
- Route selection (driver): the driver must be able to select a bus company and a specific route (an ordered list of stops) before broadcasting.
- Live map display (passenger): the passenger's screen must display the driver's current position as a moving marker on a live map, along with the passenger's own position.
- Route search (passenger): the passenger must be able to search for buses travelling between a chosen "From" stop and "To" stop, matched against the currently active route.
- QR-based boarding and exit: the passenger must be able to scan a QR code pasted on the bus to record a boarding event (capturing GPS location and time), and scan again on exit to close the trip.
- Fare calculation: on exit, the system must calculate the straight-line distance between the boarding and exit locations and apply a tiered fare rate, deducting the result from the passenger's wallet balance.
- Wallet balance: the system must track a per-device wallet balance and reject a fare deduction if the balance is insufficient.

ii. Non-Functional Requirement

- Real-time performance: location updates broadcast from the driver must reach connected passengers within a few seconds to feel "live".

- Availability: the backend must remain reachable from any network once deployed, without requiring the driver's and passenger's devices to share a local network.
- Usability: the driver and passenger interfaces must be simple enough to operate with minimal instruction — a single toggle for the driver, and one QR scan action for the passenger.
- Portability: the client application must run on Android smartphones using a single Flutter codebase.
- Data consistency: a QR scan for “exit” must only be processed if a corresponding “board” event is already open for that device, preventing duplicate or out-of-order fare calculations.



Figure 2.1: Use Case Diagram

The use case diagram illustrates two primary actors, Driver and Passenger, interacting with the BusAm system. The Driver actor selects a company and route and starts broadcasting live GPS coordinates. The Passenger actor views the live map, searches for a route between two stops, and performs QR scans to board and exit a trip, which triggers automatic fare calculation and wallet deduction.

2.1.2 Feasibility Study

i. Technical Feasibility

The system is built using widely supported, open-source technologies: Flutter/Dart for the mobile client and FastAPI (Python) for the backend, communicating over WebSockets and HTTP. Packages used — geolocator for GPS access, flutter_map with OpenStreetMap tiles for mapping, mobile_scanner for QR scanning, and web_socket_channel for real-time communication — are all mature, actively maintained Flutter packages. No proprietary or licensed SDKs are required, making the system technically feasible to build and maintain with standard developer tools.

ii. Operational Feasibility

The Driver interface requires only a single toggle (“Go Online”/“Go Offline”) after selecting a route, and the Passenger interface requires only opening the app and scanning a QR code, making the system usable without prior training. Since public bus drivers already operate smartphones for other purposes (such as ride-hailing platforms), operating a similar broadcast-toggle app is a realistic expectation.

iii. Economic Feasibility

The project uses entirely open-source and free-tier tools: Flutter and FastAPI are free and open source, OpenStreetMap tiles are free to use, and the backend is hosted on Render’s free web-service tier, which requires no payment or credit card. The only ongoing cost anticipated for a production deployment would be mobile data usage on the driver’s device, which is minimal since only small JSON coordinate payloads are transmitted every few seconds.

iv. Schedule Feasibility

The project was developed iteratively within the sixth-semester project timeline, with the core GPS tracking feature completed first, followed by route modelling, the QR-based fare system, and cloud deployment in successive stages, allowing a working system to be demonstrated at each milestone rather than only at the final deadline.

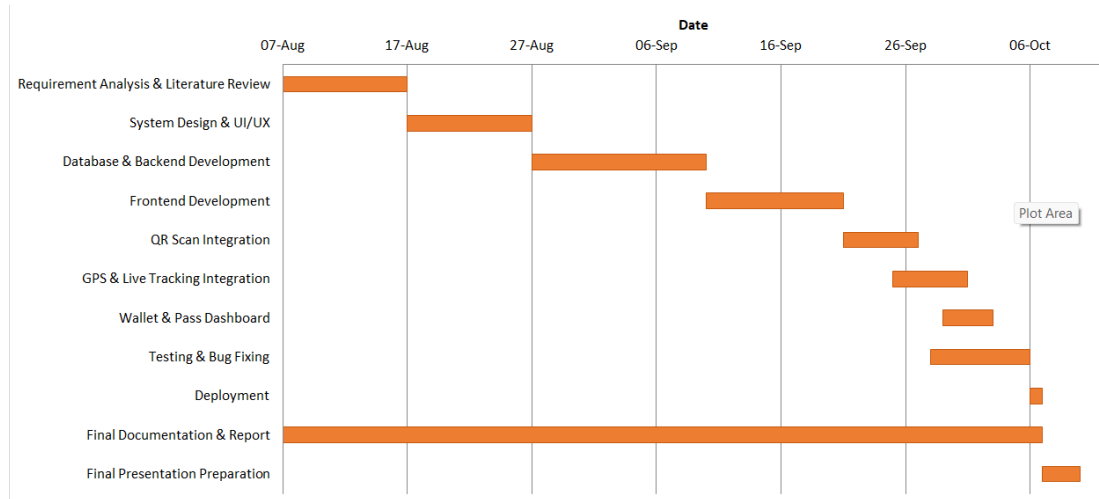


Figure 2.2: Gantt Chart

2.1.3 System Modelling

An Object-Oriented approach was used for system modelling.

i. Object Modelling using Class and Object Diagrams

The class diagram defines the BusAm architecture, centred on a BusRoute entity that groups an ordered list of Stop names under a Company. A Trip class links a device (identified by a device ID) to a boarding location, an exit location, a calculated distance, and a resulting Fare. A Wallet class tracks the balance associated with each device ID. A LiveLocation class represents the most recent broadcast position and the currently active route ID.

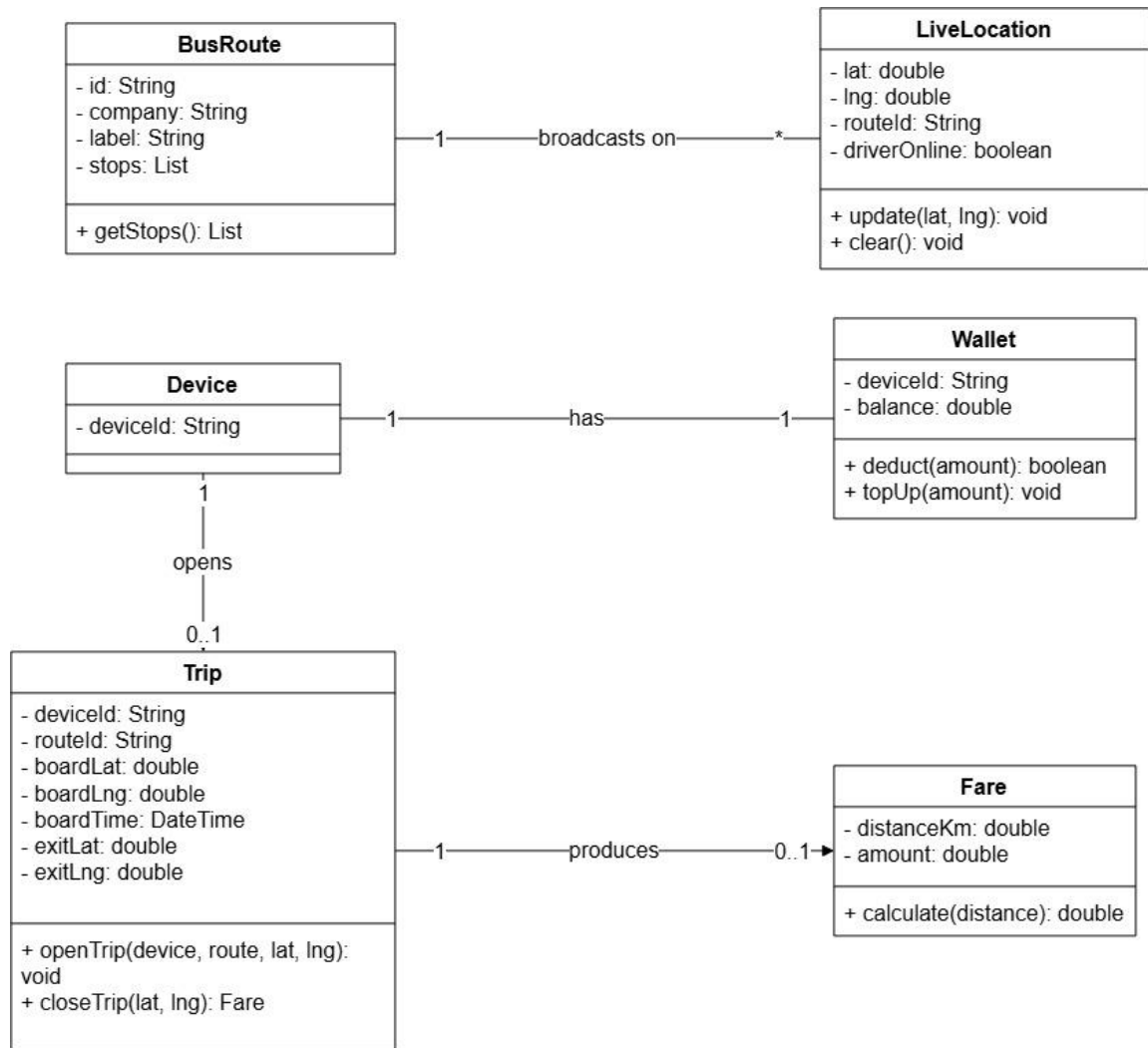


Figure 2.3: Class Diagram

The object diagram illustrates a concrete snapshot of the system: a Driver device broadcasting under route “mayuri_baudha” (Kalanki → Banasthali → Balaju → Samakhushi → Chabahil → Baudha), and a Passenger device with an open Trip that began at Kalanki and is currently awaiting an exit scan, alongside that device’s current Wallet balance.

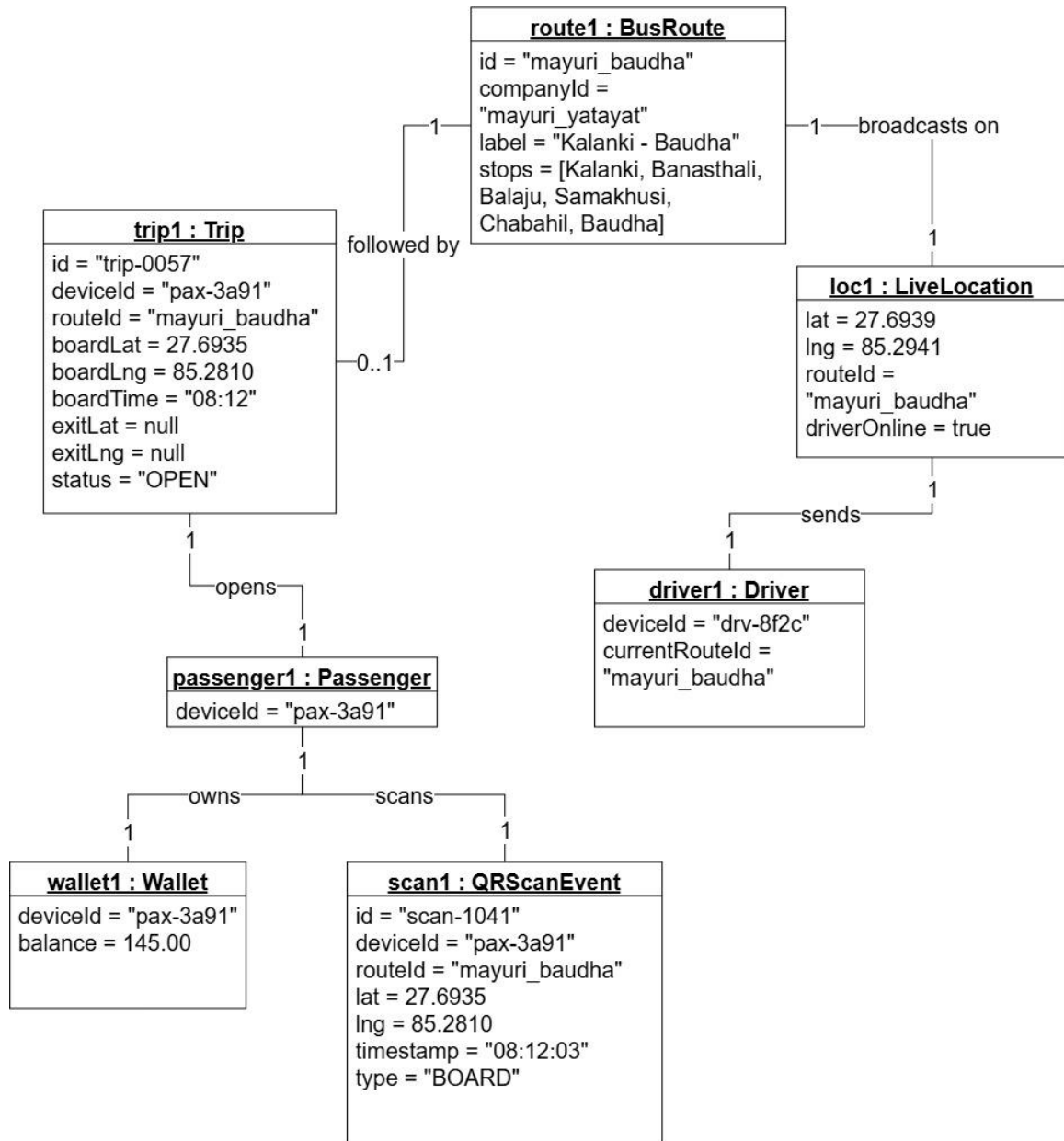


Figure 2.4: Object Diagram

ii. Dynamic Modelling using State and Sequence Diagrams

The sequence diagram below traces the driver GPS broadcasting flow: the Driver app connects to the /ws/driver WebSocket endpoint, sends a location payload (latitude, longitude, and route ID) every two to three seconds, and the backend immediately relays that payload to every currently connected Passenger app over the /ws/passenger endpoint.

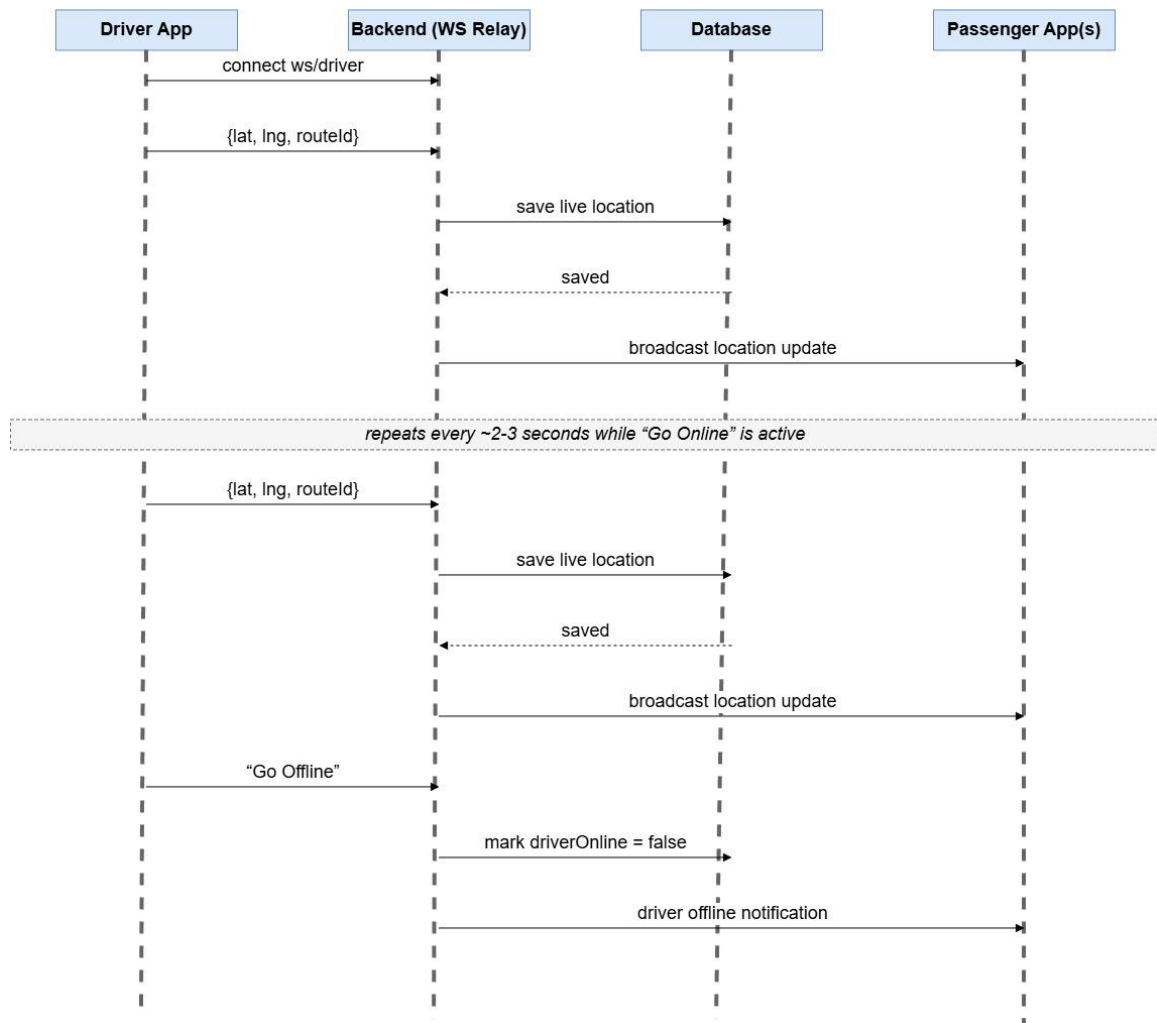


Figure 2.5: Sequence Diagram of Driver GPS Broadcasting

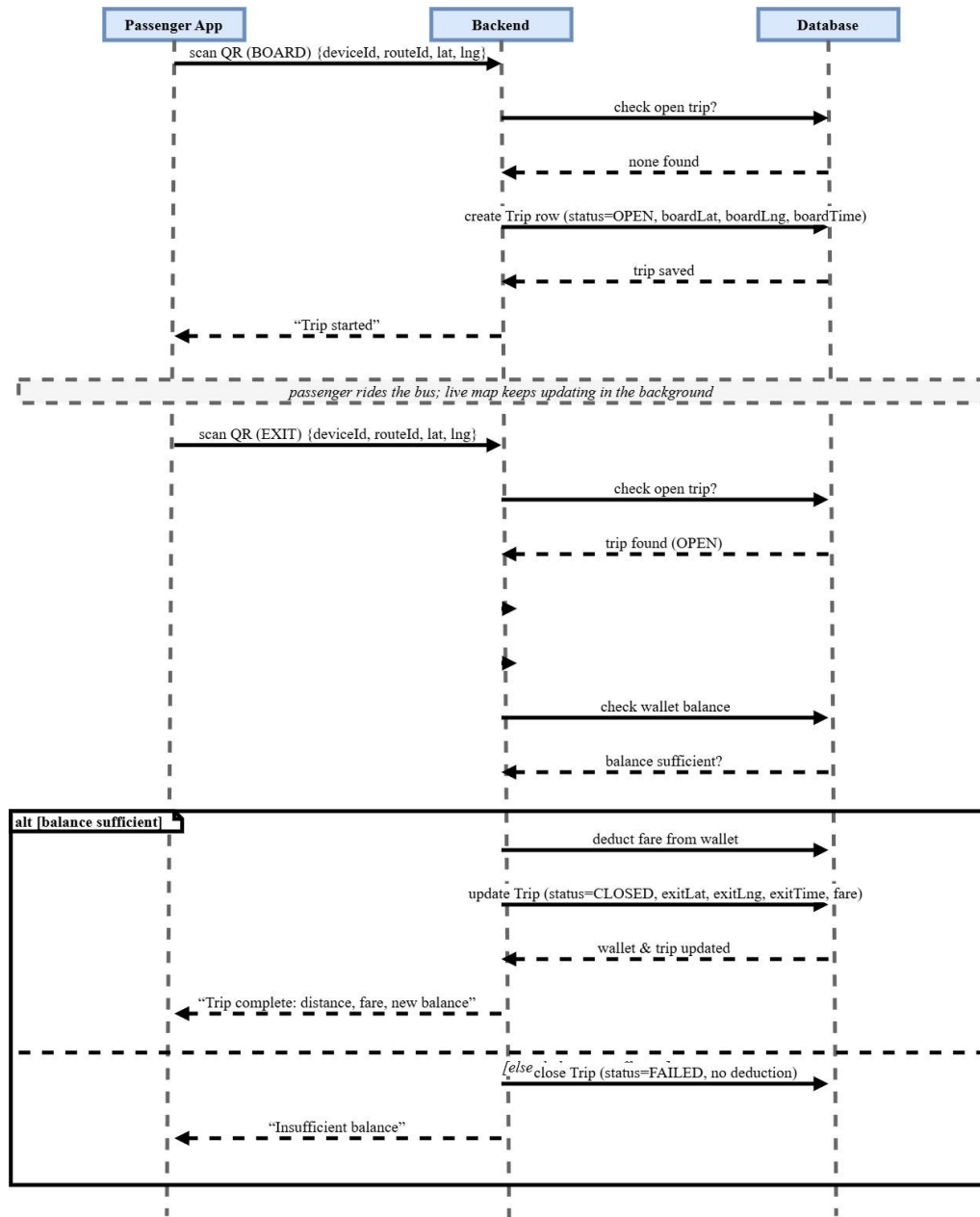


Figure 2.6: Sequence Diagram of QR-Based Boarding and Exit

The passenger state diagram shows the lifecycle of a passenger's session within the app: from Idle, to Viewing Live Map, to Searching Route, to Trip Boarded (after the first QR scan), to Trip Completed (after the second QR scan and fare deduction), returning to Idle.

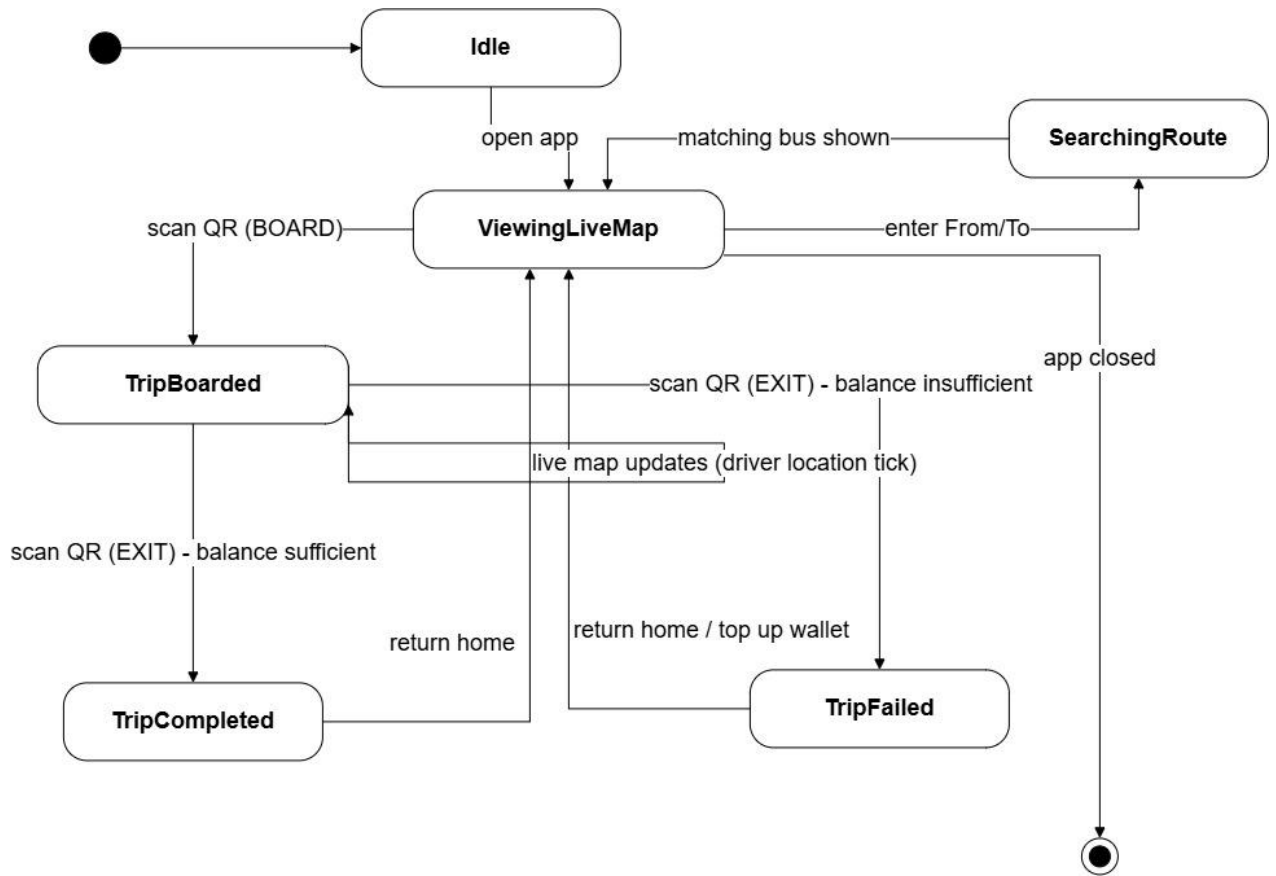


Figure 2.7: Passenger State Diagram

iii. Process Modelling using Activity Diagram

The activity diagram illustrates the overall passenger workflow: opening the app, viewing the live map, optionally searching for a route between two stops, scanning the bus QR code to board, riding the bus while the live marker updates, scanning the QR code again to exit, and receiving the calculated fare and updated wallet balance.

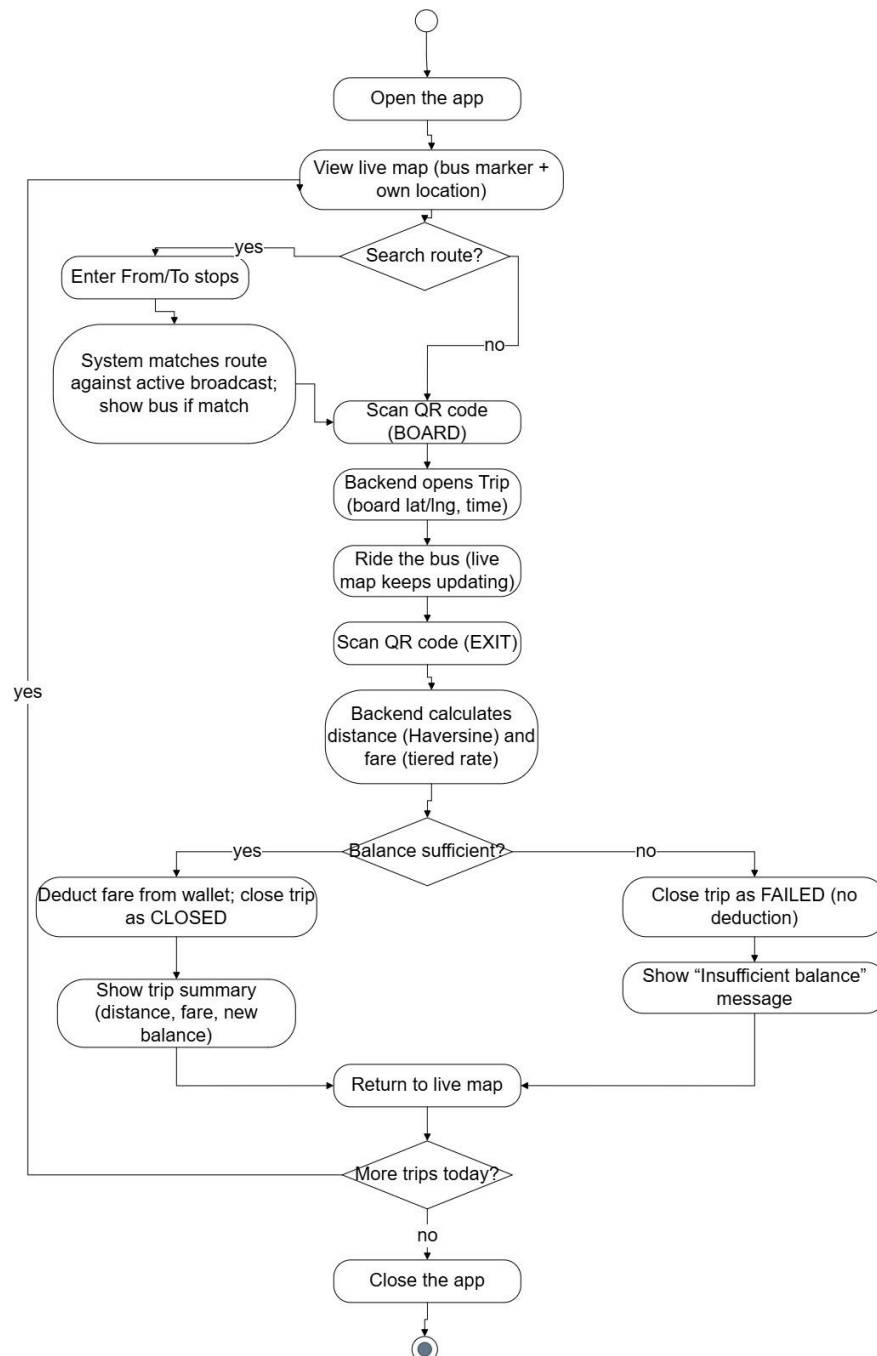


Figure 2.8: Activity Diagram of the Live Tracking and Fare Process

2.2 System Design

2.2.1 Refinement of Classes and Objects

The class diagram was refined to reflect the actual data structures implemented in code: a Dart `BusRoute` class (`id`, `company`, `label`, `stops: List<String>`) used identically on both the

driver and passenger screens; a Python-side in-memory dictionary structure for wallets ($\text{device_id} \rightarrow \text{balance}$) and open_trips ($\text{device_id} \rightarrow \{\text{route_id}, \text{board_lat}, \text{board_lng}\}$); and a latest_location dictionary ($\text{lat}, \text{lng}, \text{route_id}, \text{driver_online}$) representing the single live broadcast state.

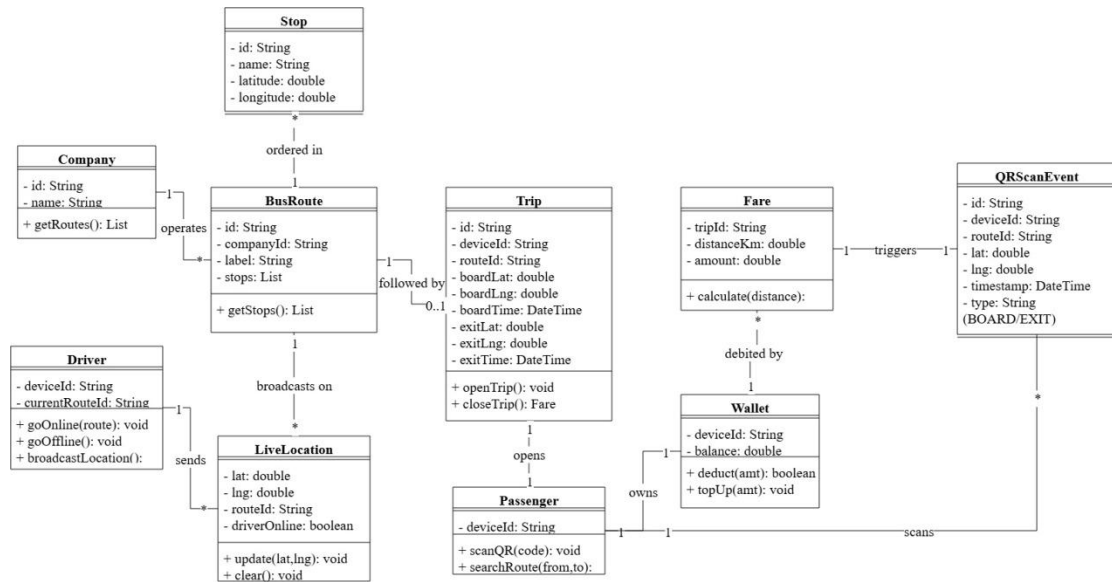


Figure 2.9: Refinement of Class Diagram

2.2.2 Component Diagram

The component diagram groups the system into a Client Layer (Driver App, Passenger App — both built with Flutter), a Communication Layer (WebSocket Relay for GPS broadcast, HTTP endpoints for QR scan and wallet operations), an Application Services layer (Route Matching Service, Fare Calculation Service, Wallet Service), and a Deployment Layer (the FastAPI application hosted on Render).

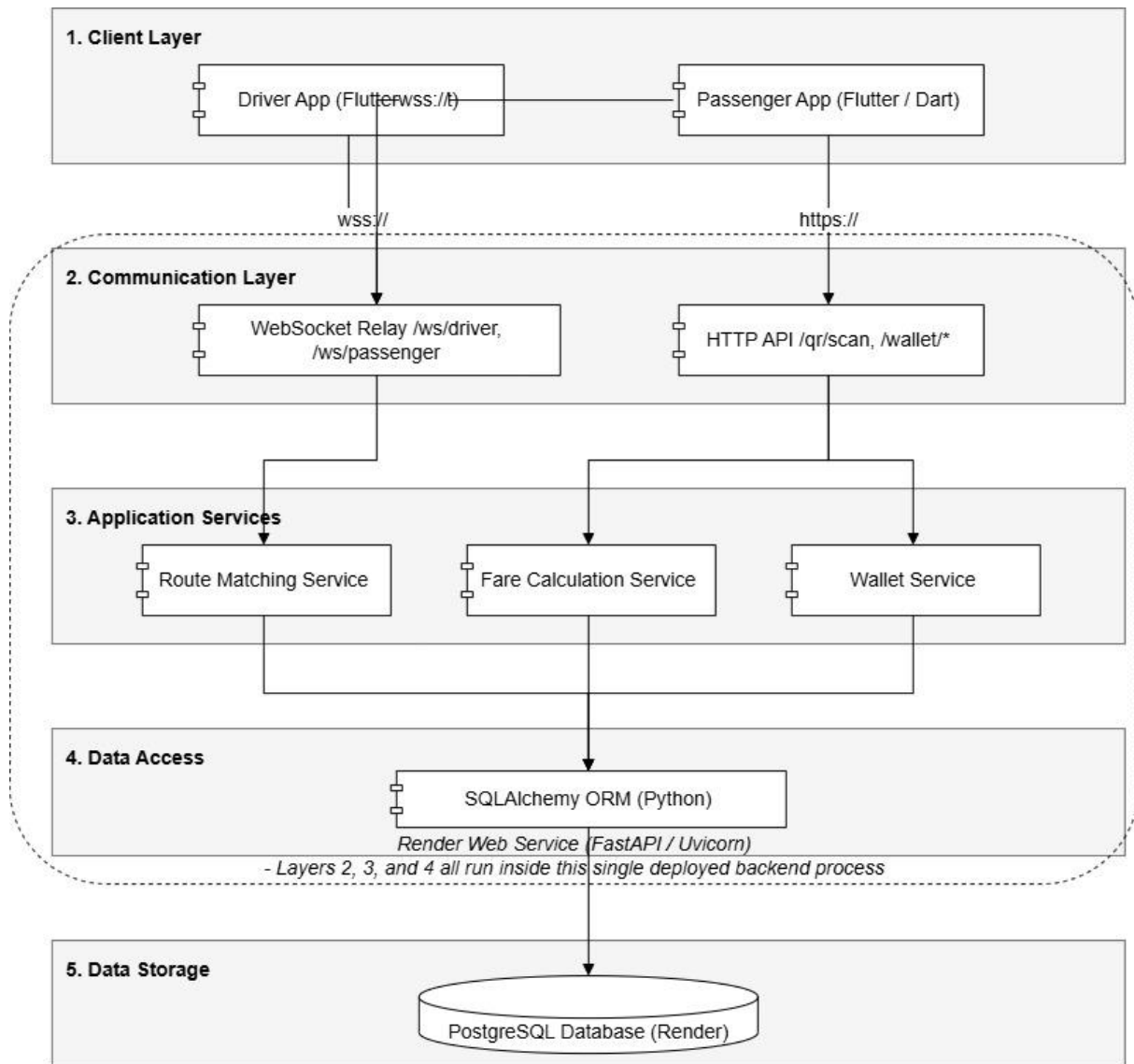


Figure 2.10: Component Diagram

2.2.3 Deployment Diagram

The deployment diagram shows the Driver and Passenger smartphones (Client Devices) communicating over the public internet (WSS/HTTPS) with the FastAPI backend deployed as a Web Service on Render. The backend process runs a single Uvicorn ASGI server that handles both the WebSocket endpoints (/ws/driver, /ws/passenger) and the HTTP endpoints (/qr/scan, /wallet/{device_id}, /wallet/load), with all state currently held in server memory rather than a separate database tier.

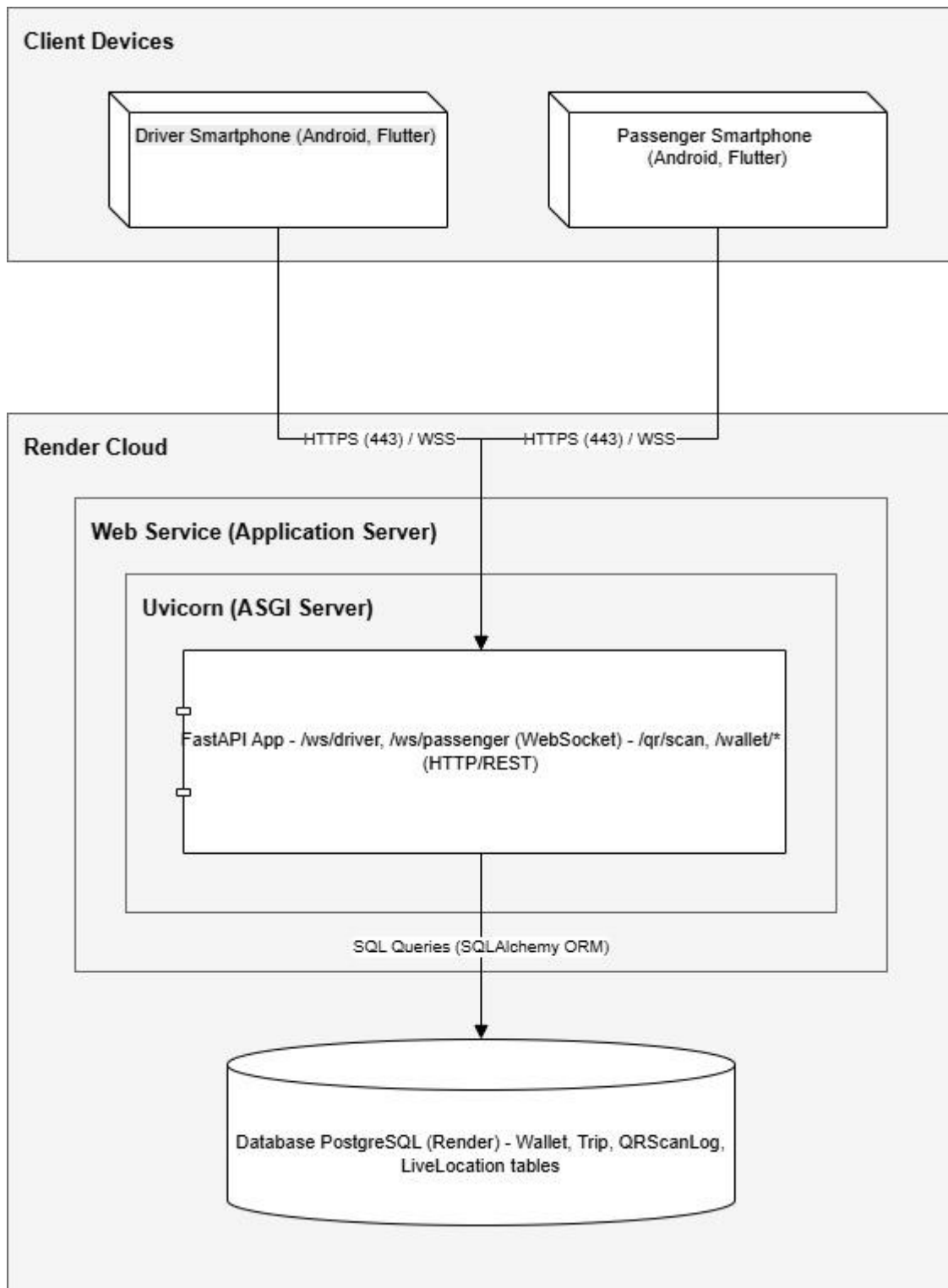


Figure 2.11: Deployment Diagram

2.3 Implementation

2.3.1 Tools Used

i. CASE Tools

- Draw.io: used to create the UML diagrams — use case, class, object, sequence, state, activity, component, and deployment diagrams.

ii. Programming Languages and Frameworks

Client Application:

- Flutter / Dart — cross-platform mobile application framework
- geolocator — device GPS access and live position streaming
- flutter_map + latlong2 — interactive map rendering using OpenStreetMap tiles
- web_socket_channel — WebSocket client for real-time communication
- mobile_scanner — camera-based QR-code scanning
- http — REST API calls to the backend for QR scan and wallet operations
- shared_preferences — local persistence of a per-device generated ID

Backend:

- FastAPI (Python 3) — web framework used for both WebSocket and REST endpoints
- Uvicorn — ASGI server used to run the FastAPI application
- Python's math module — used to implement the Haversine distance calculation

iii. Deployment and Hosting Platform

- Render — free-tier cloud web service hosting used to deploy the FastAPI backend with a fixed public URL
- GitHub — version control and the source repository connected to Render for continuous deployment

2.3.2 Implementation Detail of Modules

i. Role Selection Module

The application opens on a role-selection screen with two buttons, “I’m the Driver” and “I’m the Passenger”. Selecting a role navigates to the corresponding screen using Flutter’s Navigator, allowing a single installed application (and a single codebase) to serve both roles without a separate build for each.

ii. Live GPS Broadcasting Module (Driver)

On the driver screen, the user selects a bus company and then a specific route from a dependent dropdown (the list of available routes filters based on the chosen company). Tapping “Go Online” opens a persistent WebSocket connection to the backend’s /ws/driver endpoint and starts a repeating timer that reads the device’s current GPS position (using the geolocator package) approximately every two to three seconds and sends it, together with the selected route ID, as a JSON payload over the open connection.

iii. WebSocket Relay Module (Backend)

The backend maintains two WebSocket endpoints. The /ws/driver endpoint accepts a single driver connection, stores the most recently received location and route ID in a shared latest_location object, and immediately forwards that payload to every currently connected passenger. The /ws/passenger endpoint accepts any number of connections, registers each new connection in a list of connected passengers, sends the current latest_location to a newly connected passenger so they are not left blank, and removes a passenger from the list when their connection closes.

iv. Live Map and Route Search Module (Passenger)

The passenger screen renders a live map using flutter_map with OpenStreetMap tiles, showing the bus’s current position (received over the WebSocket connection) as a moving marker and the passenger’s own live GPS position as a second marker. A From/To search bar, backed by an autocomplete text field, lets the passenger search for routes whose ordered stop list contains both the chosen “From” and “To” stop in the correct order; the bus marker is only shown on the map if the currently broadcasting route matches the passenger’s search (or if no search has been performed).

v. QR-Based Fare Module

A single QR code, encoding a route ID, is pasted on the physical bus. When the passenger scans it for the first time, the app sends the device ID, route ID, and current GPS coordinates to the backend’s /qr/scan endpoint, which records this as an “open trip” for that device (boarding location and time). When the same QR code is scanned again, the backend recognises the open trip, calculates the straight-line distance between the boarding and current location using the Haversine formula, applies a tiered fare (for example, NPR 30 for 0–15 km and NPR 50 for 15–30 km), and deducts the resulting fare from the passenger’s wallet balance if sufficient funds are available; otherwise the trip is closed and an “insufficient balance” message is returned.

vi. Wallet Module

A simple in-memory wallet, keyed by a randomly generated device ID stored locally via shared_preferences, tracks each device’s balance. The wallet balance is checked and

updated during fare deduction, and a “Load Money” screen was designed to top up the balance through the eSewa payment gateway’s sandbox (UAT) environment using an HMAC-signed, WebView-based redirect flow, though this feature was not fully completed within the project timeline.

vii. Deployment Module

The backend, together with its requirements.txt dependency list, was pushed to a GitHub repository and connected to a Render Web Service, configured with the build command `pip install -r requirements.txt` and the start command `uvicorn main:app --host 0.0.0.0 --port $PORT`. This produced a fixed public URL that both the driver and passenger applications connect to (over `wss://` for WebSocket endpoints and `https://` for REST endpoints), removing the earlier requirement that both devices and the development laptop share the same local WiFi network.

2.3.3 Testing

Table 2.1: Unit Testing

ID	Scenario	Test Steps	Expected Result	Status
UT01	Haversine Distance Calculation	Call <code>haversine_km()</code> with two known coordinate pairs	Returns the correct straight-line distance in kilometres	Pass
UT02	Fare Tier Calculation	Call <code>calculate_fare()</code> with distances in each tier boundary	Correct fare (e.g. NPR 30 for ≤ 15 km) returned for each tier	Pass
UT03	Route Search Matching	Call <code>searchRoutes()</code> with a valid From/To stop pair	Returns only routes containing both stops in the correct order	Pass

The unit tests confirm that the two core calculation functions — Haversine distance and tiered fare — and the route search/matching logic behave correctly in isolation, forming the foundation for the fare and route-search features.

Table 2.2: Integration Testing

ID	Scenario	Test Steps	Expected Result	Status
IT01	Driver Broadcast to Passenger Display	Driver goes online and sends location; passenger app is connected	Passenger map marker updates to the driver’s new position within a few seconds	Pass
IT02	Route-Filtered Search	Passenger searches a From/To pair not matching the active broadcasting route	Bus marker and route are hidden from the search results	Pass
IT03	QR Board-then-Exit Flow	Passenger scans QR to board, then scans again at a different location	Backend returns calculated distance, fare, and updated wallet balance	Pass

ID	Scenario	Test Steps	Expected Result	Status
IT04	Insufficient Balance Handling	Passenger exits a trip with a wallet balance lower than the calculated fare	Trip closes with an “insufficient balance” response and no negative balance	Pass

The integration tests validate that the driver, backend relay, passenger, and QR-fare modules work correctly together end to end, including edge cases such as an insufficient wallet balance.

Table 2.3: System Testing

ID	Scenario	Test Steps	Expected Result	Status
ST01	Cross-Network Live Tracking	Driver and passenger devices on two different networks (WiFi and mobile data) after backend deployment	Live tracking works correctly without requiring a shared local network	Pass
ST02	Rapid Repeated QR Scans	Passenger holds the camera on the QR code for several seconds	Only the first detection is processed; subsequent detections are ignored via a scan lock	Pass
ST03	Driver Disconnection Handling	Driver goes offline mid-session	Passenger app is notified the driver is offline and the bus marker is cleared	Pass
ST04	Late-Joining Passenger	A passenger connects after the driver has already been broadcasting	Passenger immediately receives the last known bus location instead of a blank map	Pass

The system tests confirm that BusAm behaves correctly under realistic, full-system conditions — including the deployed, cross-network scenario that reflects real-world usage — and that edge cases such as repeated scans, mid-session disconnection, and late joining are handled gracefully.

CHAPTER III: CONCLUSION AND RECOMMENDATION

3.1 Summary

BusAm is a Flutter and FastAPI-based system that brings real-time GPS bus tracking and a QR-code-driven digital fare model to Kathmandu’s public bus network. The project delivers a working driver/passenger live-tracking loop over a WebSocket relay, a route-modelling and search feature built on ordered stop lists, a distance-based automated fare calculation using the Haversine formula, an in-memory wallet system, and a backend deployed to a public cloud host so the system works across different networks rather than only on a shared local WiFi connection.

3.2 Conclusion

The project successfully demonstrates that a real-time, GPS-based tracking and automated fare system can be built for a fixed-route public bus network using only widely available, open-source mobile and backend technologies, without requiring specialized transit hardware such as dedicated smart-card readers. By reusing the same route-modelling data both for driver route selection and passenger route search, and by driving fare calculation directly from GPS coordinates captured at QR-scan time, the system keeps its architecture simple while still solving the three core problems identified in the problem statement: live visibility of a bus’s location, the ability to search for a bus travelling a given path, and automated digital fare collection. The current in-memory, single-bus implementation is sufficient to demonstrate and validate this approach, while leaving clear next steps for turning it into a production-ready, multi-bus system.

3.3 Recommendation

The following recommendations are proposed for future enhancement of the system:

Short-term

- Integrate a persistent database (such as PostgreSQL) to store wallet balances, trip history, and route definitions so that data survives server restarts.
- Complete the eSewa “Load Money” payment flow so that passengers can top up their wallet directly within the app.
- Add basic user authentication (login/registration) so that a wallet is tied to a verified account rather than only a locally generated device ID.

Medium-term

- Extend the system to support multiple buses and multiple simultaneous drivers on different routes, rather than a single active broadcast.

- Build an administrative interface for managing bus companies, routes, and stops, removing the need to hardcode route data in the client application.
- Replace straight-line (Haversine) fare distance with actual road-route distance for more accurate fare calculation.

Long-term

- Introduce estimated time of arrival (ETA) predictions based on historical and live GPS data.
- Add offline trip-data caching on the driver's device so that GPS points logged during a signal drop can be synced once connectivity is restored.
- Explore a native digital pass/subscription system alongside the existing pay-per-trip wallet model.

REFERENCE

FastAPI. (2026). FastAPI framework, high performance, easy to learn, fast to code, ready for production. Retrieved from <https://fastapi.tiangolo.com/>

Flutter. (2026). Flutter – Build apps for any screen. Retrieved from <https://flutter.dev/>

OpenStreetMap Foundation. (2026). OpenStreetMap. Retrieved from <https://www.openstreetmap.org/>

Render Services, Inc. (2026). Render: Cloud application hosting for developers. Retrieved from <https://render.com/>

Pathao Nepal. (2026). Pathao – Ride, food, and courier services. Retrieved from <https://pathao.com/>

APPENDIX

